CiT Backend Engineering Track

Week 9: Spring Boot Core — IoC & Dependency Injection

The IoC Container, Beans, Dependency Injection & Configuration

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 2 — Specialisation (Weeks 9–20)
Week	Week 9
Instructor	Jim & Anderson
Duration	6 months | 24 weeks
Tech Stack	Java 25, Spring Boot 4, PostgreSQL, IntelliJ IDEA, Gradle

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

We thank You for carrying us through the foundation. Today we begin to go deeper — from simply using our tools to truly understanding how they work. Give us the patience to sit with harder ideas, the humility to admit what we do not yet grasp, and the perseverance to press on until understanding comes.

As Hebrews 6:1 urges us, "leaving the discussion of the elementary principles… let us go on to perfection." Lord, mature us as engineers and as people, that our growing skill would always be matched by growing character.

May we be diligent students who do not just copy but truly understand. May we encourage one another when we struggle. And may everything we build be offered as service to others.

In Jesus' name we pray,
Amen.

Scripture for the Week

"Therefore, leaving the discussion of the elementary principles of Christ, let us go on to perfection."

— Hebrews 6:1 (NKJV)

Table of Contents

1. Welcome to Week 9 — Phase 2 Begins

Congratulations — you have completed the Foundation Phase. You can write Java, design with objects, store data in a database, build a REST API, and integrate a full-stack product. Phase 2, Specialisation, is where you move from competent beginner toward professional engineer.

For the next four weeks we go deep on Spring Boot — the framework you have been using since Week 7. And here is the shift that defines Phase 2: until now you have mostly used tools; now you will understand how they work. That difference is what separates someone who can follow a tutorial from someone who can build and debug real systems.

We start with the single most important idea in the entire Spring framework: Inversion of Control and Dependency Injection. Every annotation you sprinkled on your Week 7–8 API — @RestController, @Service, the constructor that magically received a repository — is powered by the machinery we uncover today.

What to Expect in Week 9

By the end of this session, you will be able to:

- Explain the problem of tight coupling and why creating your own dependencies hurts
- Describe Inversion of Control (IoC) and what the Spring container does
- Define what a bean is and what the ApplicationContext holds
- Use constructor-based Dependency Injection correctly, and say why it beats field injection
- Register beans with @Component/@Service/@Repository and with @Configuration + @Bean
- Externalise settings with application.yml, @Value, @ConfigurationProperties, and Spring Profiles
- Refactor an API so every layer is a cleanly injected Spring bean

2. The Problem: Tight Coupling

To appreciate why Spring exists, you must first feel the pain it removes. That pain is called tight coupling — when a class creates and depends directly on the concrete objects it needs.

2.1 A Class That Builds Its Own Dependencies

Imagine a controller that needs a service, which in turn needs a repository. The naive approach is to create each one with new:

```
public class StudentController {

    private StudentService service = new StudentService();
    // ...and inside StudentService:
    //     private StudentRepository repo = new StudentRepository();
}
```

It works — but it is rigid. The controller is now permanently welded to that exact service, and the service to that exact repository.

2.2 Why This Hurts

Problem	Why It Matters
Hard to change	To swap in a different or improved service, you must edit every class that said new StudentService().
Hard to test	You cannot substitute a fake/mock service in a unit test — the controller always builds the real one, which may hit a real database.
Hidden wiring	Each class secretly constructs its own world. There is no single place that describes how the app is assembled.
Duplicated setup	If the repository needs a database connection, every creator must know how to build it correctly.

The Core Insight

A class should **declare what it needs** — not decide how to build it. Deciding "which service, built how" is someone else's job. In Spring, that someone is the container. This one shift in thinking is the doorway to everything else this week.

3. Inversion of Control & the Spring Container

The solution to tight coupling has a name: Inversion of Control, or IoC. It is the beating heart of Spring.

3.1 What "Inversion of Control" Means

Normally, your code is in control: it decides when to create objects and wires them together with new. IoC inverts that. You hand control of object creation and wiring over to the framework. Your classes simply announce what they need, and the framework supplies it.

The old Hollywood line captures it: "Don't call us — we'll call you." You do not go and fetch your dependencies; the container brings them to you.

3.2 The Spring Container

At the centre of every Spring application is the container. When your app starts, the container:

1. Discovers all the classes you have marked as Spring-managed
2. Creates one instance of each (an object it now owns)
3. Works out which ones depend on which, and injects them into each other
4. Hands you fully assembled, ready-to-use objects

The Concierge Analogy

Think of the Spring container as a hotel concierge. You do not go into the kitchen to cook, drive to the airport for a taxi, or call the laundry yourself. You simply state what you need, and the concierge — who knows how to obtain everything and keeps it all organised — delivers it. Your job is to do your work; the concierge's job is to provide what that work requires.

3.3 Why This Is Better

Once the container owns object creation, all the pain from Section 2 melts away: swapping an implementation is a one-line change, tests can inject fakes freely, and there is exactly one place — the container — that assembles the whole application.

4. Beans & the Application Context

Now we name the pieces precisely. Two terms come up constantly in Spring, and they are simpler than they sound.

4.1 What Is a Bean?

A bean is simply an object that the Spring container creates, owns, and manages. That is the whole definition. Your StudentService, your StudentRepository, your controller — once Spring manages them, each is a bean.

The word carries no magic. "Bean" just means "an object Spring is in charge of," as opposed to an object you created yourself with new.

4.2 What Is the ApplicationContext?

The ApplicationContext is the container itself — the object that holds every bean and knows how they fit together. When Spring Boot starts (remember SpringApplication.run(...) from Week 7), it builds the ApplicationContext, fills it with your beans, and wires them up.

Term	Plain English
Bean	An object Spring creates and manages for you.
ApplicationContext	The container that holds all the beans and injects them where needed.
Bean Scope	How many instances Spring keeps. The default, singleton, means one shared instance for the whole app.

4.3 Singletons by Default

By default every bean is a singleton: Spring creates exactly one instance and shares it everywhere it is needed. This is efficient and almost always what you want for services and repositories, because they hold behaviour, not per-user data.

5. Dependency Injection in Practice

Dependency Injection (DI) is how the container gives a bean the other beans it needs. "Injection" simply means the dependency is handed in from outside, rather than created inside.

5.1 Constructor Injection — The Right Way

You declare what you need as a constructor parameter. Spring sees the parameter, finds the matching bean, and passes it in when it builds your object.

```
@Service
public class StudentService {

    private final StudentRepository repository;

    // Spring calls this constructor and injects the repository bean
    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }
}
```

The class never says `new StudentRepository()`. It simply asks for one, and the container provides it. This is the pattern you used in Week 7 — now you know why it works.

5.2 Why Constructor Injection Beats Field Injection

You may see older code inject directly onto a field with `@Autowired`. Prefer constructor injection.

Constructor injection	Field injection (@Autowired on a field)
Dependencies can be final — guaranteed set once and never null	Fields cannot be final; can be left unset
Trivial to test — just pass a fake into the constructor	Hard to test without Spring or reflection tricks
Required dependencies are obvious from the signature	Hidden dependencies scattered through the class

You Often Do Not Need @Autowired At All

If a Spring-managed class has exactly one constructor, Spring injects its parameters automatically — you do not even need to write @Autowired. Clean, quiet, and correct.

6. Defining Beans — Stereotypes, @Configuration & Component Scanning

For the container to manage a class, you must tell it the class is a bean. There are two ways, depending on whether the class is yours.

6.1 Stereotype Annotations (for your own classes)

Mark your own classes with a stereotype annotation. They all register the class as a bean; the different names simply document the class's role and make code easier to read.

Annotation	Use It On
@Component	Any Spring-managed class (the generic stereotype).
@Service	A class holding business logic (the service layer).
@Repository	A data-access class. Also translates database exceptions.
@RestController	A web-layer class that handles HTTP requests (from Week 7).

6.2 Component Scanning

How does Spring find these annotated classes? Component scanning. On start-up, Spring scans the package of your main @SpringBootApplication class and every sub-package, registering every stereotyped class it finds as a bean. This is why your project's package structure matters — keep your classes under the main application package.

6.3 @Configuration and @Bean (for classes you do not own)

You cannot add @Component to a class from a third-party library — you did not write it. Instead, declare it as a bean inside a @Configuration class using an @Bean method.

```
@Configuration
public class AppConfig {

   @Bean
   public RestClient restClient() {
      return RestClient.create();   // Spring now manages this object
   }
}
```

The method returns the object; the @Bean annotation tells Spring to keep it in the ApplicationContext and inject it wherever a RestClient is needed.

7. Externalised Configuration & Profiles

Professional applications never hard-code settings like database URLs, passwords, or feature flags. Those live outside the code, so the same build can run in development, testing, and production without change. Spring makes this effortless.

7.1 application.yml

Spring Boot reads settings from src/main/resources/application.yml (or .properties). You met it briefly in Week 7; here is a fuller picture.

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/cit_school
    username: postgres
    password: ${DB_PASSWORD}      # read from an environment variable

cit:
  max-students-per-class: 40      # a custom setting of our own
```

7.2 Reading Settings — @Value and @ConfigurationProperties

Pull a single value into a bean with @Value:

```
@Value("${cit.max-students-per-class}")
private int maxStudents;
```

For a group of related settings, bind them to a typed class with @ConfigurationProperties — cleaner and type-safe:

```
@ConfigurationProperties(prefix = "cit")
public class CitProperties {
    private int maxStudentsPerClass;
    // getters and setters
}
```

7.3 Profiles & Environment Variables

A profile is a named set of settings for one environment. Keep application-dev.yml and application-prod.yml, and activate one with spring.profiles.active=prod. Secrets like passwords should come from environment variables (as ${DB_PASSWORD} above), never be committed to your repository.

Never Commit Secrets

Database passwords, API keys, and tokens must never appear in your source code or be pushed to GitHub. Read them from environment variables. A leaked secret in a public repo is one of the most common — and most damaging — security mistakes a junior engineer can make.

8. Putting It Together — A Cleanly Wired API

Let us see every idea from this week working together in the Student API you already know. Notice that not one class uses new to build a dependency — the container assembles everything.

8.1 The Three Layers as Beans

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long> { }
```

```
@Service
public class StudentService {
    private final StudentRepository repository;
    public StudentService(StudentRepository repository) {
        this.repository = repository;   // injected by Spring
    }
    public List<Student> findAll() { return repository.findAll(); }
}
```

```
@RestController
@RequestMapping("/api/students")
public class StudentController {
    private final StudentService service;
    public StudentController(StudentService service) {
        this.service = service;         // injected by Spring
    }
    @GetMapping
    public List<Student> getAll() { return service.findAll(); }
}
```

8.2 What Happens at Start-Up

When the app boots, the container scans these classes, creates one bean of each, then injects the repository into the service and the service into the controller — a fully assembled chain, ready to serve requests. You wrote three focused classes; Spring wired them into an application.

Exercise: Add a Layer, Change Nothing Else

Add a new @Service called GradeCalculator with a method that turns an average into a letter grade. Inject it into StudentService through the constructor and use it.

- Confirm you never wrote new GradeCalculator() anywhere
- Notice the controller did not change at all — the wiring is the container's job
- Write a plain unit test for StudentService by passing fake dependencies into its constructor — no Spring needed

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- Tight coupling — building your own dependencies with new — makes code rigid and hard to test
- Inversion of Control (IoC): the framework, not your code, creates and wires objects
- A bean is an object Spring manages; the ApplicationContext is the container that holds them
- Constructor injection is the correct way to receive dependencies — and why it beats field injection
- Registering beans with stereotypes (@Service, @Repository) and with @Configuration + @Bean
- Component scanning — how Spring discovers your beans
- Externalising settings with application.yml, @Value, @ConfigurationProperties, profiles, and env vars — and never committing secrets

9.2 Homework

5. Complete the in-class exercise and push it to GitHub
6. Refactor your Week 7–8 API so every layer is a Spring bean wired by constructor injection, with no new for any dependency
7. Move at least two settings (e.g. a page size and a greeting message) into application.yml and read them with @ConfigurationProperties
8. Preview Week 10: Read about Spring MVC request mapping and DTOs, and skim a REST API response-envelope convention. Come with one question ready.

Coming Next — Week 10: Spring MVC & Clean API Design

Jim & Anderson will go deep on the web layer: flexible request mapping, request/response DTOs, shaping responses with ResponseEntity and status codes, and a consistent API response envelope — the professional standards that make an API a pleasure to consume.

Closing Reflection

"...rooted and built up in Him and established in the faith, as you have been taught."

— Colossians 2:7 (NKJV)

This week you stopped memorising annotations and started understanding the framework beneath them. Roots you cannot see are what hold a great tree upright in the storm. Build your understanding deep, not just wide, and what you make will stand.

Glossary of Terms

Term	Definition
ApplicationContext	The Spring container that holds and wires all the beans in an application
Bean	An object created, owned, and managed by the Spring container
Component Scanning	Spring's start-up search that finds stereotyped classes and registers them as beans
Constructor Injection	Receiving dependencies as constructor parameters — the preferred DI style
Dependency Injection	The container supplying a bean with the other beans it needs, from outside
Inversion of Control	Handing control of object creation and wiring from your code to the framework
Profile	A named set of configuration for one environment (e.g. dev, prod)
Singleton Scope	The default bean scope — one shared instance for the whole application
Stereotype	An annotation (@Component, @Service, @Repository, @RestController) marking a class as a bean
Tight Coupling	A class depending directly on concrete objects it creates itself, making change and testing hard
@Bean	A method in a @Configuration class that produces a bean Spring should manage
@ConfigurationProperties	Binds a group of external settings to the fields of a typed class
@Value	Injects a single external configuration value into a field